When Does Admission Control Pay? The Memory Tier Sets the Cost of Concurrency in Agentic LLM Serving

Anonymous Submission

***Abstract*—An agentic LLM session is a long, multi-round job whose KV footprint grows every round, each carrying a first-token deadline; on a memory-constrained accelerator the pool binds, and admission control—which rounds to admit—looks like the lever. Busy-time service rate is $\overline{R}/\tau$, so raising concurrency by f buys $f^{1-\epsilon}$, ϵ the elasticity of per-step *service time* to the running count—the step, not the request, whose completion time carries queueing and step count. On a memory-bound agentic replay $\epsilon = 0.38$ to 0.61, and it tracks the memory tier, not the controller—an association across five policies, not a demonstration. With a host tier four static points hold 87.9–92.4% of the reuse bound over $\overline{R}_{\text{busy}}$ 1.25 to 1.75; without one, realization falls from 92.5% to 36.4%, as recomputation. The verdict is negative: nine controller variants spanning accounting, growth reserve and execution granularity, two more on the budget ceiling, and an independent congestion-based controller reimplemented—none above the static admission baseline for the same trace, deadline and resource budget, the best twice-measured at -0.30 and $+0.22$ pp of deadline satisfaction rate, both on the line: no improvement beyond the ± 1.0 pp band fixed in advance. A tenth arm—the same controller revision, one expression changed—lands above it twice in three runs, by 1.3 and 2.2 pp, the third on the line: both readings are the *release* rule's, not the admission decision's. A second GPU and a second trace reproduce the direction, not the level. Our criterion is a diagnostic: it neither returns the verdict nor explains the failure. Miss rates are measured, not a latency bound.**

***Index Terms*—LLM serving, admission control, KV cache, agentic workloads, performance characterization, negative results**

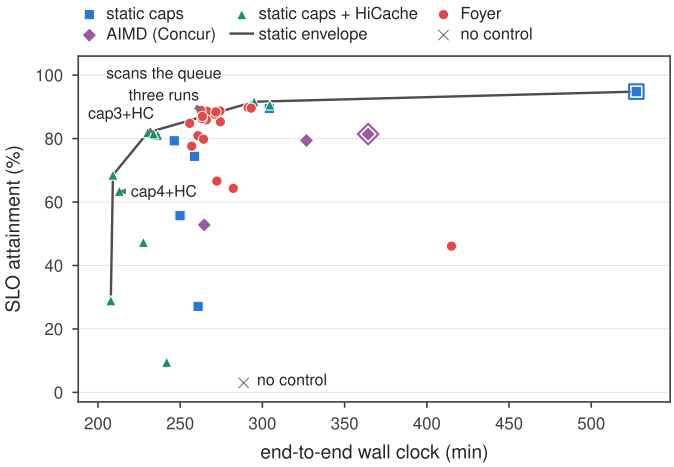


Fig. 1: Every completed run on the (wall clock, SLO) plane. The line is the upper-left envelope of the static points, drawn from the data, so it uses each configuration's *better* run where a repeat exists. Two points are drawn with a hollow ring: both were measured while other engines shared the host, and neither shows aggregate inflation in its per-request decode interval (§VIII). Verdicts are at the 10 s deadline used throughout; §VIII reports how far they depend on it.

I. Introduction

An agentic LLM session is not a request. It is a long-lived job that issues a round of inference, waits on a tool, and comes back—tens of times—with a context that grows every round [1]–[3]. On a memory-constrained accelerator, where every round carries a first-token deadline [4], admission control—which rounds to admit—looks like the obvious lever: the KV pool is the binding resource [5]–[7], a single large session can occupy most of it, and sessions are heterogeneous.

We believed this, built a controller that does exactly that, and spent a considerable amount of GPU time discovering that on our workload and hardware it is false. That negative result is most of what we have.

A. What we measured

We serve a replay of 200 real coding-agent sessions on a single RTX 3090 (Qwen2.5-Coder-7B, 96K context, SGLang, a 101,432-token KV pool) and compare admission policies against a sweep of static concurrency caps. Nine variants of our own controller span three mechanistically distinct ideas, a fourth is tested by two further arms, and a reimplementation of an independent congestion-based controller [8] runs alongside them; an all-throttles-off ablation runs a different controller and is not counted. None of the nine is above the static Pareto front—the static admission baseline for the same trace, deadline and resource budget (Fig. 1). Its best, now measured twice, reads -0.30 and $+0.22$ pp of deadline satisfaction rate (first tokens, timed from the engine; §II)—either side of the line, both inside the ± 1.0 pp band we call a tie, so we record it as on the line: no improvement outside that band over the static baseline. One further arm lands above it on two of its three runs—the same controller revision, one expression changed, its third on the line (§V). Four of the nine were measured while other jobs shared the host; we re-measured all four, which moved them 11.2, 6.5, 6.4 and 4.5 points closer to the front without crossing it.

B. Why

We decompose the wall clock into *busy* time (the engine has at least one request in flight) and *idle* time (none). Busy time is governed by $W_{\text{busy}} \approx (G/\overline{R})\,\tau$ (Eq. 3)—$\overline{R}$ the mean running requests, G the generated tokens, τ the per-step cost—and admission control moves $\overline{R}$ and little else. But τ is not constant in $\overline{R}$: across our runs τ_1 spans 27.0–36.7 ms, and on the device-only family raising the busy-time running count from 1.00 to 1.84 absorbs a third of the busy time the extra concurrency would otherwise have saved (§III).

The idle component is structural rather than a control failure: a session in a tool wait issues no request [9], so when all live sessions happen to be waiting the engine is genuinely empty. The stretches are long in the tail—66.1 to 69.5% of all idle *time* sits in stretches over 20 s, an order of magnitude above the 2 s control cycle—and the demanded median inter-round gap is 0.34 s while the *mean* is 15.8–16.9 s, with 18% of those transitions wait more than ten seconds.

C. Contributions

- We identify the memory tier as what determines, in our measurements, how much of the achievable prefix reuse survives added concurrency, separating the *local* return on added concurrency from whether a controller wins *end to end*, where idle time and the SLO axis also move (§VI, §IV); a second workload with the two conditions associated with room still does not win.
- We test four routes by which an adaptive admission controller was expected to beat a tuned static cap—two falsified against criteria fixed before the runs, one a knob rather than a hypothesis, the fourth falsified by an argument about what the permit mechanism cannot express—and we do not call the last two falsifications of a stated hypothesis, because they were not.
- We give a two-part decomposition of the wall clock (§III) that locates the cancellation, and reduce the question to one measurable number (§IV), the elasticity of per-step *service time* with respect to the running count—of the step, not of the request—offered as a diagnostic rather than a predictor, and §IV is explicit about the sense in which it is circular.
- We make the measurements reproducible from the raw per-step logs (§VIII) and report the seven claims we had to retract.

D. What we do not claim

We do not claim admission control is useless in general: we observe no gain beyond the band *here*—a gain inside it is not excluded—and we show the frontier measurement that decides that; the diagnostic of §IV is a cheaper check an engineer can run before committing to a sweep, not a verdict on its own. Nor do we claim our hardware and model are representative—one 7B model on two accelerators of different sizes. On the larger card, where we have since run a controller and extended the static sweep to meet it, the controller lands 0.1 pp *above* the static front at its own wall clock, inside the tie band (§VI). We read that as a second configuration *not overturning* the negative result, and deliberately not as the result reproducing at full strength: it is *one* controller run, and our pre-registered criterion for that tier does not cover it.

II. Background and Setup

A. Workload

We replay 200 coding-agent sessions drawn from the TraceLab corpus [1]. Each session is a sequence of rounds; a round presents a prompt of median 23,187 tokens (p90 51,064; maximum 89,436, which alone is 88% of the KV pool), and between rounds the session waits on a tool before returning with a longer context. Arrivals are Poisson at $\lambda = 0.04\,\text{s}^{-1}$ over an 80-minute window, giving 999 steps in total. Two sessions of maximum size cannot be co-resident: their combined contexts (89,436 and 87,821 tokens) exceed the pool by 75%.

The tool wait is the load's defining property and the one most easily misread. Its demanded *median* is 0.34 s, which invites the conclusion that sessions never leave the engine; its demanded *mean* is 15.82 s and its maximum is 300 s, and 18.0% of the 799 round transitions wait more than ten seconds. Our replay harness honors the trace: on the run with no controller in the path the measured gap is 15.82 s mean against a demanded 15.82 s, and the median of the per-transition measured/demanded ratios is 1.001. The controller's own run measures 16.85 s, and we attribute the 1.03 s difference to the controller's admission delay rather than to the harness.

B. Engine and the binding constraint

We use SGLang v0.5.10.post1 [10] with Qwen2.5-Coder-7B-Instruct [11] extended to 96K context, on one RTX 3090 (24 GB). KV is paged [5]; at `mem-fraction-static` 0.85 the pool is 101,432 tokens. Hierarchical caching (HiCache), with a host tier of twice the device pool [6], [12], is enabled on the configurations marked *+HC* and on every Foyer run of this trace; the five plain static caps run without it. We keep both because the host tier turns out to change the mechanism (§III). Every run is guarded: a pool that does not come out at $101{,}432 \pm 20$ voids the run rather than recording it.

C. The controller under test

Foyer is an admission controller that reads engine telemetry and issues *named* permits rather than a scalar cap [13]: a session is admitted by appearing in a permit file, and the runner refuses to start a round for a session not listed. This is what allows per-session actions at all, including the pause primitive of §V-D: the controller never frees capacity by discarding a session, only by admitting fewer, and that is a statement about the *policy*, not the memory (§III).

Each control cycle (2 s) the controller computes

$$\Pi \;=\; \max(\rho, \textstyle\sum_i c_i) \;+\; \sum_i g_i, \tag{1}$$

where ρ is the engine's reported occupancy, c_i the context length of live session i, and g_i its forecast growth over the next `horizon` rounds. A session is admitted when $\Pi + n \le B$, with n the candidate's estimated need and B the budget:

a measured physical occupancy, a reservation for sessions currently parked in a tool wait (whose KV is evictable and therefore absent from ρ), and a reservation for future growth.

D. Baselines and the verdict criterion

Baselines are static caps: at most N sessions resident, $N \in \{1, \ldots, 5\}$, with and without HiCache. We compare end-to-end wall clock (first submission to last completion) against the deadline satisfaction rate [14] (SLO attainment: the fraction of requests whose time-to-first-token is under the 10 s deadline; the miss rate is one minus it). That clock is the engine's and per request—it starts when the request is submitted *to the engine*—so a permit wait outside the door is not charged, the reading most favourable to a controller that imposes one; §VIII reports the ready clock, on which it is charged and no run is above the line. We report measured miss rates, not a latency bound [15].

"Faster than cap-N" is not the question: the question is whether a policy reaches a point *above* the static front, because mixing cap-N and cap-M by time slice approximates any point between two static caps. We flag the segment argument as an argument rather than a curve—carrying queue state and cache contents across a switch could bend the line—and we have measured it once: a run that alternates the cap between three and two every 60 s on the main trace lands 0.89 pp *below* the line at its own wall clock, inside the tie band and on the below side (`slice23.csv`)—the direction the argument needs, since that is not where an overstated line would show. It is the stricter of the two criteria, which makes our negative result *easier* to reach, not harder.

Our best arm reads -0.30 and $+0.22$ pp against the line, and $+5.8$ and $+6.8$ pp against the nearest static anchor below it in wall clock (`cap3`+HiCache's repeat; $+6.6$ and $+7.6$ against that anchor's *first* run—so the sign of the comparison is not in question, but its size moves with which run and which repeat you take). The arm whose repeats bracket the line most widely is a different one—`horizon`=1, landing at 0.93, 5.58 and 3.22 pp below it across its three runs—and we name the run we mean whenever we quote a distance for it, because the spread across those runs (4.4 pp of SLO) is wider than the margin that decides our headline.

We take the upper-left envelope of the static points and report each controller's signed distance above or below it: more than $+1.0$ pp is above the front, less than -1.0 pp below it, and the band between the two is *on* it—a tie, not a win: no distinguishable improvement observed. The band is a convention we fixed before running anything, not a calibrated confidence bound. Our later replicate measurements bracket it rather than justify it: the SLO spread of configurations we ran more than once ranges from 0.5 pp to 5.1 pp, so for some configurations it is generous, for others far too narrow. We flag this as a weakness of the criterion, not a defence of it.

One property of the envelope deserves flagging: one row in Table I's controller section—the all-throttles-off arm, which runs a different controller—finishes slower than 295 minutes, judged against the right-hand segment from `cap2`+HiCache at 295 minutes to `cap1` at 527, and `cap1` is the only anchor there that runs *without* HiCache, so it is an interpolation between one host-tier point and one device-only point. We report the conservative reading: *below the front as we defined it*, not *below every static configuration*.

III. Decomposing the Wall Clock

A. Two components

We split the wall clock at every sampled instant by whether the engine has any request in flight:

$$W = W_{\text{busy}} + W_{\text{idle}}. \tag{2}$$

Fig. 2 shows the split for the seventeen of the thirty-nine runs of Table I's families we have per-step logs for. Idle time is not noise: it spans 6.3% to 41.9% of the sampled span, and 6.4% to 42.5% against the request wall clock of Table I, and it is not spread evenly—over 2,374 stretches the mean is 20.6 s and 17.7% of them exceed 20 s, accounting for 69.5% of all idle *time*, the tail a scheduler feels. We report the mean and the share rather than the median, which is not robust to which runs enter the pool: it reads 10.0 s without `cap1` and 5.0 s with it. The mean (18.9–20.6 s) and the over-20 s share (66.1–69.5%) move by little across the four variants in `idle_pooled.csv`—two distinct pools—and the claim that the tail dominates does not depend on the choice.

The two components move independently and in opposite directions, so neither is a proxy for the wall clock. The no-control baseline (§V) cuts idle time by 13.8 minutes against the base configuration—it admits everything, so nothing queues—and finishes only 2.7 minutes sooner, since its busy time grows by 11.1 minutes; the trade costs it the SLO column outright, 89.8% down to 3.0%. The arm that disables every throttle moves the other way, and further: idle *rises* by 129 minutes while busy *falls* by 5.4, so the 124 minutes it adds are almost entirely waiting, bought with 43.6 pp of SLO. Neither example is one of the damaged runs of §VIII—both are clean in the co-tenancy audit—and neither belongs to the controller family whose clean step-cost range that section quotes.

B. What determines busy time

During busy time the engine executes steps; each step produces one token per running request. Hence [16]

$$W_{\text{busy}} \approx \frac{G}{\overline{R}}\,\tau, \tag{3}$$

with G the total generated tokens (fixed by the trace at 706,010) and τ the per-step cost—not a fit parameter we tuned: we obtain it two independent ways.

a) Method 1 (derived).: $\tau_1 = W_{\text{busy}} \cdot \overline{R}/G$, with $\overline{R}$ the mean running count *during busy time*—the busy-conditional mean throughout, since the whole-run mean includes the zero-count idle samples and would make τ_1 short by the *idle* fraction, 36.3% for `cap1`, whose busy share is 63.7%.

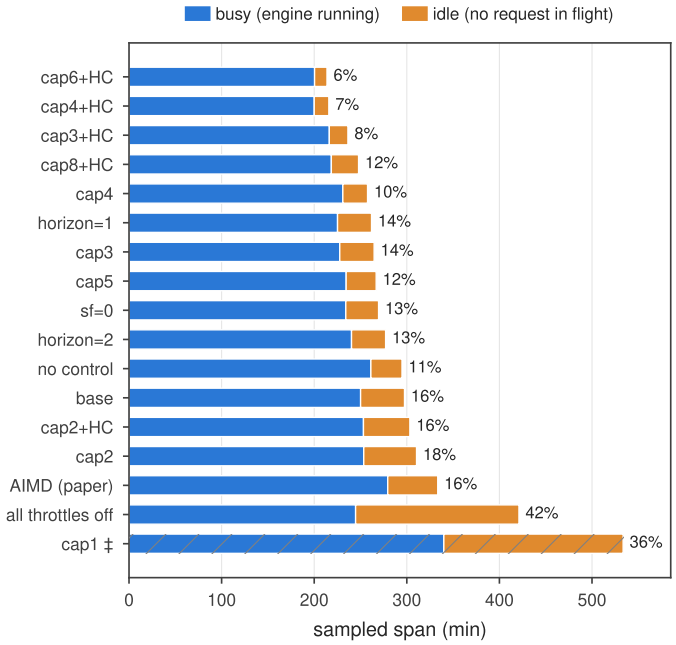


Fig. 2: Busy and idle time for the seventeen runs the figure draws, ordered by total duration; the idle range (6.3% to 41.9%; 6.3% for the fastest configuration, 36.3% for the slowest) is computed over all thirty-nine runs. One bar is hatched—the same ‡ flag as in Table I—measured while other engines shared the host, with no measurable damage (§VIII).

b) Method 2 (independent).: The median per-request decode interval $(\text{total_duration} - \text{first_token})/(\text{output_len} - 1)$ over steps with output_len $\geq$ 32: an independent cross-check that does not depend on the quantity it explains, and the one we quote for the cost of a decode step. The two disagree—τ_1 exceeds τ_2 on every run we accept, by 8.4–39.7% over the 38 left once the repeat excluded below is dropped. The gap is *consistent with* engine steps that are prefill and therefore emit no output token—we can bound it, not identify it, because subtracting a median from a mean does not isolate a component: it is a ratio of *times*, not of steps, and we have no measurement of the fraction of step *counts* that are prefill, and even as a time ratio it is the gap between two measurements, not a decomposition of one—so we cannot rule out that some of the difference is the two estimators disagreeing for other reasons. They are therefore not two estimates of one number but measurements of two different things, and we do not size a component with their difference. (The `horizon`=1 repeat is excluded from the reuse accounting: its token counters are anomalous, so its τ columns, reuse and recomputation are unusable, while its wall clock and SLO—from the client—are ones we use.) The engine's own latency counters are exported as zero by SGLang v0.5.10.post1, so no directly measured step time exists in our data, and the step cost we quote is inferred rather than read [17].

C. The per-step cost is not constant in concurrency

Equation 3 would be uninteresting if τ were a constant. It is not: across the fourteen runs we fitted, τ_1 ranges from 27.0 to 36.7 ms and τ_2 from 20.6 to 31.2 ms, and both rise with the running count—fitting Eq. 4 to the five device-only configurations gives $\varepsilon = 0.38$ for τ_1 and 0.61 for τ_2 (Fig. 3). The relation is a trend, not a law: `cap4_hc` runs 1.73 requests at 29.8 ms while `horizon`= 0 runs 1.58 at 30.2, so running count sets a floor and a trend, not a value, and on the host-tier family, as we show below, not the value at all.

This is the crux of the paper. Hold τ_1 at its `cap1` value of 28.8 ms and busy time falls as $1/\overline{R}$: raising the running count from 1.00 to 1.84 should have cut busy time by a factor of 1.84, from 340 minutes to 184, but the measured curve flattens and lands at 234. The rise in τ takes back 50 of the 156 minutes the extra concurrency would otherwise have bought—*a third* of the gain, not all of it and, as an earlier revision claimed, not more than half; the shortfall compounds with every controller that tries to exploit the difference.

a) What the engine is doing when it is not decoding.: Across the five static caps *without* HiCache, tokens evicted from the radix cache [18], [19] and prompt tokens that had to be prefilled rather than reused track each other almost exactly—an evicted prefix is recomputed, close to one token for one, and that recomputation is chunked into decode steps—and recomputation rises with τ across four of the five points, the exception being `cap1`. It is five points on five policies, not an intervention—we did not hold concurrency fixed and vary eviction—so we call it consistent, not demonstrated.

b) Where the mechanism stops.: With HiCache the correspondence breaks in a way that confirms the mechanism rather than undermining it: `cap4_hc` misses 22.4 M tokens on the device, more than the device-only `cap4`, but only 9.1 M of that is recomputed and the other 13.3 M is served from the host tier, which across the thirty host-tier runs serves 3.1–19.3 M each. Recomputation is still what the device cannot absorb; the host tier is a second absorber in front of it [20].

D. What tracks reuse is the tier, not the policy

The device-miss column hides the quantity that matters: *reuse* is the prompt tokens served from anywhere in the memory hierarchy—device radix cache plus host tier—so reuse + recomputed = prompt, fixed at 28.5 M by the trace. The bound on reuse has to be defined independently of what we observed, or the argument is circular: the planned reusable prefix over all 999 rounds of `replay_pois200_l04.csv` totals 21,540,371 tokens against a prompt of 28,526,871, so no policy can reuse more than 75.5%. It is a bound rather than a description—no configuration we ran reaches it, the best realizes 92.5%—and how much of the bound is realized is where the tier enters.

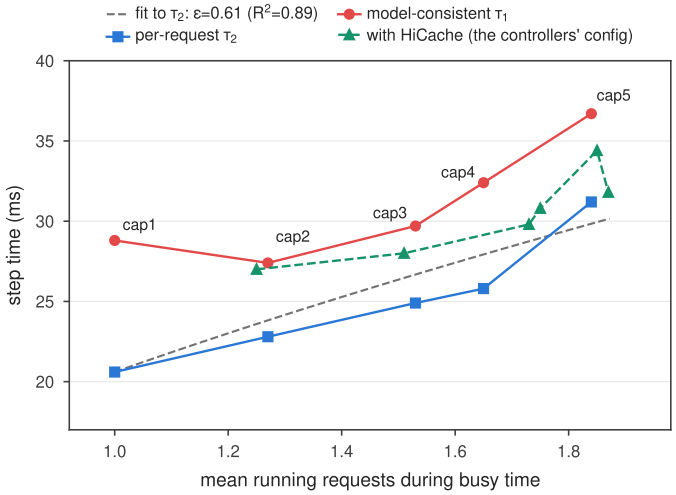


Fig. 3: Per-step cost against the running count during busy time: τ_1 (circles) and τ_2 (squares) for the five device-only configurations, and τ_1 (triangles) for the six HiCache configurations the controllers run. Over $\overline{R}_{\text{busy}}$ 1.25 to 1.73 the host-tier curve rises only 27.0 to 29.8 ms while the device-only curve over a comparable span rises 27.4 to 36.7, and τ_1 exceeds τ_2 by 17.7–39.7%; beyond 1.73 it climbs to 34.4 ms at 1.85, where realization begins to fall too. The fitted exponent is the device-only elasticity of Eq. 4.

device-only	$\overline{R}_{\text{busy}}$	1.00	1.27	1.53	1.65	1.84
	realized (% of bound)	92.5	89.7	79.0	60.7	36.4
	recomputed (M)	8.6	9.2	11.5	15.4	20.7
host tier	$\overline{R}_{\text{busy}}$	1.25	1.51	1.73	1.75	
	realized (% of bound)	92.4	91.3	90.2	87.9	
	recomputed (M)	8.6	8.9	9.1	9.6	

Device-only, realization falls from 92.5% to 36.4%, the shortfall being recomputation; with a host tier it holds at 87.9–92.4% on the four static host-tier caps above and at 59.7–92.5% across the thirty host-tier runs that completed this trace at a *single* configuration (HiCache on, at least 995 of its 999 steps; the one run that alternates its cap, §II, is not one of them), the lowest `cap8`+HC at 59.7%. The *device* misses more where we can pair them (`cap3`+HC 26.0 M against `cap3`'s 11.5 M) while the host serves 3.1–19.3 M in front of the recomputation, but the ordering is *not* universal—at a cap of five the host-tier run's device misses 19.0 M against the device-only run's 20.7 M—and we report the reversal with no mechanism for it. Five runs meeting the predicate are left out for reasons that are not mechanical: one whose prompt-token counter reads 1.41× the trace plan with neither client-side retries nor retractions to account for the excess, and four superseded measurements we re-ran clean (§VIII). (75.5% is the trace's bound as a share of the prompt, and 92.5% a share of *that bound*; `reuse.csv` carries both.)

Device occupancy does not explain the split: `cap3_fixed` and `cap3_hc` run at nearly the same occupancy (55.8% and 55.5%) and running count (1.53 and 1.51) yet realize 79.0% and 91.3% of the bound—occupancy is instantaneous while reuse depends on history, a session's prefix having to survive from one round to the next, and we do not have the measurement that would separate those. The device-only side is one run there, but a repeat has since finished: the gap becomes 9.8 points rather than 12.2 and the side is no longer a single sample, its two runs differing by 2.4 pp (2.4299 before rounding, as with every other gap here). A pre-registered back-to-back pairing of the two tiers on one card, its criterion written down in advance, has since finished: *it lands at* 10.1 *points—a pass by* 0.14, against a threshold fixed at 10 points, its runs differing by 0.8% in mean occupancy and 3.1% in running count, both inside the 10% tolerance the criterion requires before it will read the comparison. The pair is not a paired experiment—same harness generation, about a day and two GPUs apart—and pairing the repeat with the host-tier run removes the card half of the confound, though those two remain three and a half days and a generation apart (§VIII states what we did to rule out co-tenancy).

A capacity-controlled version—same trace and policy, pool cut to half or three quarters—does not return a realization figure: a step whose prompt exceeds the pool is never executed, the engine answers with a single token, and the runner records that as a success, so the failure counter stays at zero and the loss is invisible in every counter we have. The rule is exact rather than statistical, in one direction without exception—every step reporting *one prompt token* while planning tens of thousands has a prompt larger than the pool (six runs at four pool sizes, no counterexample), though not the same set as *steps that generate one token*, which clean runs contain too. The full pool (101,432 tokens) holds the trace's longest prompt (89,436), so the control marks a boundary rather than a point on the curve, and the comparison is bounded by the trace, not by the system; it shows capacity matters, but capacity is not occupancy, so it does not separate the two accounts either. The claim this section supports is the narrow one: within a configuration the host tier preserves realization at a matched running count and a matched occupancy, and without it realization falls—a correlation we can name, on a pair confounded in the two ways above. We do not treat the arithmetic as a result: it assumes KV demand grows linearly in the running count, which heterogeneous sessions need not satisfy.

a) What is left unexplained.: Four of the arms above—the resident base, `horizon= 0`, `horizon= 2` and the `horizon= 1` no-hw arm—were *first* measured while three other engine instances shared the host, which inflates per-step cost by 12–41% and leaves the volume columns untouched (§VIII); all four have since been re-measured alone, and the values above are the clean ones. (The flagged runs are superseded and not in the table; the labels name the arms.)

E. Why idle time exists, and why recovering it does not pay

Some idle is structural: a session in a tool wait issues no request, and an admission policy sets how many sessions are live, so an individual stretch is the workload's and how much a *run* accumulates is not. The obvious remedy—keep more

sessions live so that someone is always ready—is what `bres` does, and the clean measurement of it says the remedy works and does not pay: what it does not do is improve the criterion (§V). Idle is recoverable in part; recovering it is not what pays.

IV. A Criterion for When Admission Control Can Pay

Throughput during busy time is $\overline{R}/\tau$ [21], so raising concurrency by a factor f buys a factor $f^{1-\epsilon}$, where

$$\epsilon \;=\; \frac{\mathrm{d}\ln\tau}{\mathrm{d}\ln\overline{R}} \tag{4}$$

is the elasticity of per-step cost with respect to the running count. At $\epsilon = 0$ the step cost is flat, concurrency converts one for one into throughput, and admission control has room; at $\epsilon = 1$ the two cancel exactly and it has none.

On the five device-only configurations, where §III establishes that the step cost is driven by recomputation, $\overline{R}_{\text{busy}}$ runs from 1.00 to 1.84. A least-squares fit of Eq. 4 over them gives $\epsilon = 0.61$ on the per-request measurement ($R^2 = 0.89$), or 0.38 on the model-consistent one (Fig. 3). In the units a controller works in, adding 10% concurrency buys 3.8% to 6.1% of busy-time throughput, against the 10% it would buy at $\epsilon = 0$.

The segment elasticities are $\epsilon = -0.21$ from `cap1` to `cap2`, $+0.43$ from `cap2` to `cap3`, $+1.15$ from `cap3` to `cap4`, and $+1.14$ from `cap4` to `cap5`. Below $\overline{R}_{\text{busy}} \approx 1.5$ added concurrency still buys something; above, ϵ passes one and busy-time throughput stops rising with the running count. That is not the same as the run getting slower end to end, since less idle time can more than pay for a longer busy time. The single number 0.38 averages those regimes and describes none of them.

The controllers run in a different region: all nine controller arms run with HiCache enabled, which the five device-only caps do not. (Every elasticity here, fitted or segmented, uses $\overline{R}_{\text{busy}}$ at the two-decimal precision of `decomp.csv`; at six decimals the *fitted* values read 0.37 and 0.60, and the segments move by 0.01–0.03.) On the three HiCache configurations that span the controllers' range, the same elasticity gives $\epsilon = +0.19$ from `cap2`+HC to `cap3`+HC and $+0.46$ from `cap3`+HC to `cap4`+HC, over $\overline{R}_{\text{busy}}$ 1.25 to 1.73—the range covering the whole controller operating region (1.35 to 1.69), over which ϵ stays below one. `cap5`+HC then adds a segment with $\epsilon = +2.87$ whose denominator is near zero: its running count moves only from 1.73 to 1.75 while the step cost moves by 1.0 ms, and `cap6`+HC at $\overline{R}_{\text{busy}}$ 1.87 gives $\epsilon = +0.48$. We do *not* read the $+2.87$ segment as "the spike was an artifact": both step-cost measurements agree something real happens there, but the small denominator costs us *resolution*, not the existence of a cost difference, and we do not read the difference as an elasticity in the running count. The per-request τ_2, which shares no code path with τ_1, gives $\epsilon = +4.3$ there and rises monotonically across all six points (22.7, 24.5, 25.7, 26.8, 27.6, 29.0 ms). What the sweep does show is the weaker thing we can support: the flat regime does not continue indefinitely—`cap8`+HC realizes 59.7% of the bound at the far end of the swept range—but our six points cannot say where the regime ends.

What the measurement does is rule one explanation out; it does not establish the opposite (§V), and the failure is not that this workload denies admission control its lever, but that we could not build a rule that uses it well.

What this does and does not establish: the HiCache curve rests on six configurations and five segments—four computable, the fifth $\overline{R}$-reversed and therefore given no coefficient—one run each, with no error bars, stopping at 1.87—so the cliff question is partly answered rather than merely unasked, since realization does start to fall at the far end of the swept range. Each run is the first of its configuration, whereas §V's frontier uses the repeats where a pair exists; substituting them moves the first two segments from $+0.19$ and $+0.46$ to $+0.20$ and $+0.44$, which changes nothing we say, and it *reverses the third*, because the repeat of `cap4`+HC sits at a slightly *higher* running count than `cap5`+HC, so that segment's $\overline{R}$ interval runs backwards and its ϵ comes out at -5.2. We report the third segment and read neither a value nor a location off it. It is measured on different policies rather than by an intervention—a cross-policy association, not a transportable causal derivative. On our data it is also computed from the same wall clock the model then explains—τ_1 is derived from W_{busy}, so using it to predict W_{busy} restates rather than tests—while τ_2, genuinely independent of the wall clock, measures only decode and cannot be dropped into a throughput equation that includes prefill.

V. Results: Four Routes, One Outcome

A. The frontier

Fig. 1 places every completed run on the (wall clock, SLO) plane and draws the upper-left envelope of the static points; Table I gives the numbers and each controller's distance from it. The result is the same for the nine policy variants of the controller we deployed, and all nine are clean measurements: *none* of their runs is above the envelope at the 10 s threshold. The *best* is `horizon`=2, run twice: -0.30 and $+0.22$ pp, either side of the line and both inside our ± 1.0 pp band. (Best, not closest: `single-flight off`'s repeat reads $+0.13$, nearer the line, but its reported run is -0.55.) Against the discrete static points rather than the envelope between them, that arm is *not dominated*: every static configuration faster than it scores lower on SLO, and every one that scores higher is slower. We take the envelope as the baseline, and §II states what that choice costs us.

Two points in Fig. 1 are above the line, and they are one arm: two of its three rows of Table I, `scans the queue` and its repeat, neither of them one of the nine. All three are the same controller revision and flags as the base's own repeat, with exactly one expression changed: that revision releases only the *head* of the waiting queue unless single-flight is explicitly disabled, and this arm scans the whole queue and releases the first session that fits it. Their logs show the change

to be narrow—one admission per cycle, never two—and the source differs in that expression plus a comment block; the launcher differs too and we cannot diff its earlier revision (§VIII). The three measure $+1.30$, $+2.16$ and $+0.29$ pp: two clear the band by 0.30 and 1.16 pp, the third does not. Against the 0.5 to 4.6 pp that separates repeats of a single controller configuration in our set (the spread of Δ; `horizon`=2's own two runs are the tightest of the five groups at 0.5 pp) the arm's own three span 1.87 pp—more than the band's 1.0 pp half-width and straddling its edge, its first and third 1.01 pp apart—so two of three, not a stable crossing. The first's wall clock is 26.9 minutes (9.2%) below the base's repeat at 1.0 pp lower SLO. The other clock, which charges the arm for the permit wait it imposes, leaves none of the three above the line: -0.56, $+0.96$ and -1.07 pp. Nor is it simply that scanning the queue wins—one of the nine, `single-flight off`, scans it too, landing 0.55 pp *below* the line on its first run and $+0.13$ pp on its repeat. These establish the reach of the negative result: the readings above the line come from the *release* rule, not from the budget rule admission acts on.

The base configuration's repeats are tight: four runs spanning 0.5 pp of SLO around an 89.5% mean—though not one build, since one is a later controller revision and one ran on the other card, so 0.5 pp is the spread we observed, not a clean estimate of one revision's noise. Against the 3.5 pp separating it from `single-flight off` (89.8% against 86.3%; §VIII says which run)—several times its own noise, and a trade rather than a win: 28 minutes of wall clock for 3.5 pp of SLO.

Four of the nine carried the co-tenancy flag of Table I and were re-measured alone (§VIII); we do not read them as agreeing—one contamination window, not four independent replications—and even the largest correction leaves its arm below the line.[1]

The no-control arm is the other end of the front: every session admitted the moment it arrives finishes in 288.6 minutes at 3.0% SLO, 87.6 pp below the envelope, with a median time-to-first-token of 1,774 seconds.

a) Notes to Table I.: The published-parameter AIMD row is the third attempt, the first whose wall clock is usable. Twenty-three further runs on this trace are in no table of ours, and where we cite any of them we name them. Two *rows of this table* stopped short of 999 (`rg4` 998 and the no-control baseline 997); our verdict script excludes runs below 999 steps, so these two are not judged against the front there, but their Δ here comes from the same full-precision coordinates as every other row. ‡ marks a run measured while three or more other engine instances shared the host for at least an hour: a risk flag, not a measurement of damage. One row now carries it, `cap1`, and five more did in earlier versions; `cap1` shows no aggregate inflation (20.6 ms), so we keep its number and say why. *The no-control arm writes no admission log, so its *Conc.* is estimated from the per-step log's session spans; on the five static caps of §III the two estimators agree to two decimals. This table omits eight of the static caps we ran. §VIII gives the audit.

[1]The repeats are uneven, and we do not want the tightest pair to stand for all of them. Nine configurations have been run more than once, their SLO spreads spanning 0.5 to 5.1 pp, and the three widest belong to the three arms that decide something, so where a small margin matters our repeats are weakest. The asymmetry runs against the controllers: the envelope takes each anchor's *better* run where a repeat exists, while most controllers are a single run. The exposure is bounded—the verdicts are set by the segment to `cap2`+HiCache, whose repeats differ by 0.8 and 1.0 pp, below the ± 1.0 pp band we judge in.

TABLE I: The runs the results below turn on. *Conc.* is the time-weighted mean number of *live* sessions, not the running-request count $\overline{R}$ of §III; Δ is the signed distance from the static envelope at that wall clock (§II). The notes below define both.

Policy	Wall (min)	Conc.	SLO (%)	Δ (pp)
Static caps				
no control	288.6	94.9	3.0	−87.62
cap1‡	527.4	1.00	94.8	*anchor*
Static caps + HiCache				
cap2+HC	295.0	1.99	91.6	*anchor*
cap3+HC	236.4	2.93	81.1	−1.72
cap3+HC (repeat)	230.3	2.97	81.9	*anchor*
cap4+HC	213.0	3.93	63.4	−7.55
cap4+HC (repeat)	209.1	3.87	68.5	*anchor*
cap6+HC	207.8	5.85	28.9	*anchor*
Foyer controller variants				
scans the queue (2nd run)	262.6	2.41	88.9	+2.16
scans the queue	266.3	2.34	88.6	+1.30
scans the queue (3rd run)	271.7	2.29	88.4	+0.29
horizon=2 (2nd run)	274.2	2.22	88.7	+0.22
single-flight off (2nd run)	263.5	2.44	87.0	+0.13
horizon=2	271.0	2.28	87.7	−0.30
single-flight off	263.3	2.43	86.3	−0.55
horizon=1	255.8	2.58	84.8	−0.93
horizon=3 (base)	291.3	2.06	89.8	−1.24
resident base	265.9	2.41	85.9	−1.33
horizon=3 (2nd run)	293.2	2.07	89.6	−1.73
horizon=1 (3rd run)	274.4	2.37	85.3	−3.22
horizon=1 (repeat)	260.8	2.69	80.9	−5.58
horizon=0	264.2	2.64	79.8	−7.19
horizon=1, no hw	257.0	2.92	77.6	−8.31
no gate, no resv.	272.3	3.25	66.6	−21.62
running gate=4	282.3	3.03	64.3	−25.39
all throttles off	415.0	2.58	46.1	−47.10
Independent adaptive controller (§V-E)				
AIMD, paper params	326.8	2.17	79.4	−12.65
AIMD, retuned	264.6	3.87	52.8	−34.28

B. The controllers leave memory unused

Our controller admits fewer sessions rather than discarding one, so its pool occupancy is a policy output, not a constraint: the base configuration holds a median 49.2% of the pool while 66 sessions queue for admission, and the configuration that finishes fastest is a poor fast end (`cap6`+HC, 28.9% on SLO). Occupancy and running requests track each other (`occupancy.csv`, Spearman $\rho = 0.97$), but running count does *not* order wall clock: `cap3`+HC and `horizon`=1, no hw carry the same mean running count to two decimals and finish 21 minutes apart.

The controller believes it is nearly full. It admits against a projection (Eq. 1) that runs at a median 7.6% free of its budget, while its measured occupancy at the same instants leaves 48.2% of that budget, or 50.8% of the pool, idle. The

projection is 1.79× the occupancy the engine reports, and not a constant of the design: it ranges 1.32 to 1.96 across the twenty-seven controller runs that finished on this trace. Underuse is a property of the controller's estimates rather than of the hardware, but it does not order the outcomes, so it is not the whole explanation either; and removing the single-flight gate is the same trade every other route makes—it moves that arm along the front rather than past it.

C. What the two mechanisms explain, arm by arm

Two mechanisms have been offered; we say what each one covers. For every controller arm we took the static cap closest to it in wall clock, out of every cap we ran, and compared occupancy and step cost: an arm is attributed to the unused-memory mechanism if it runs fewer than 92% of its comparison's requests, and to the step-cost mechanism if it runs a comparable number—within 8%—at a more than 8% higher τ_1; what is left is a residual, unless no comparable cap confirms the τ difference, in which case we record it as *unattributable*. The comparability clause is load-bearing.

Of the thirteen arms, seven are attributed—six to unused memory and one to step cost (the published-parameter AIMD); two have a τ no matched comparison can confirm (`rg4` and all-throttles-off); and four are residuals—`horizon=` 1, the base, `breshz1` and the retuned AIMD, whose occupancy and step cost both sit close to their comparison cap's, so their shortfall is real but we cannot say from these measurements which mechanism produced it. No row carries the co-tenancy flag, because the classification is made on τ_1, exactly the column co-tenancy inflates, and the arms it would have flagged have all been re-measured alone. Two limits on what that buys. The comparison is a static cap matched on *wall clock*, and the arms that finish near 260 minutes are matched against `cap5` of the full run table, whose SLO is 27.1%—so the classification describes an arm relative to a wall-clock peer, not to the static *front*. And the six unused-memory rows run fewer requests than their comparison, which is a description of what they did, not a demonstration that doing otherwise would have won. The per-arm readings are in `attribution.csv`.

D. Why the knobs do not help

Our controller's projection (Eq. 1) charges a session for its full context even while it is parked in a tool wait, when its KV is evictable and therefore absent from the engine's reported occupancy. We argued this was a double charge, removed it, and admitted on engine telemetry alone (*resident base* in Table I). The charge is not double. Removing it does what the argument predicted on the engine side, with no eviction spiral, and finishes 25.4 minutes earlier (265.9 against 291.3). It is nonetheless not a win: the SLO falls from 89.8% to 85.9%, and the static front itself falls 3.81 pp over those same 25.4 minutes, so the arm keeps its place relative to the envelope to within 0.09 pp while sliding 25 minutes along it, at the front's own slope. At 265.9 minutes the front sits at 87.2%, so the arm would have had to give up no more than 2.6 pp of SLO instead of 3.9, and it did not. We rest nothing on the 0.09 pp: the base configuration's own four runs sit at -1.24, -1.73, -1.49 and $+0.18$ pp against this same front, a 1.9 pp spread.

The other two knobs the mechanism names behave the same way. Removing the growth reserve entirely costs 10.0 pp of SLO, and the response is non-monotone in wall clock: one round is the fastest point, two rounds is 20 minutes *faster* than three, and none of the four beats the base on the SLO axis. The best is `horizon`=2, whose two runs read -0.30 and $+0.22$ pp; the one-round optimum, which this paper spends the most words on, does not reproduce, its two repeats landing 5.6 and 3.2 pp below the line. Lowering the budget fraction slides the point *below* the front rather than along it: budgets of 0.85 and 0.90 finish at 333.9 and 315.9 minutes for 89.9% and 87.7% SLO, -2.24 and -4.22 pp from the envelope at those wall clocks, against -0.93 and -1.24 for the two nearest arms at the 0.95 the rest of the paper uses. None of the three adds a mechanism.

A fourth variant attacks the assumption the first three share—that what a controller sets is *how many sessions are resident*. Because a live session spends most of its time parked, capping *running requests* rather than resident sessions should keep the engine busy. We implemented it as a gate on `num_running_reqs` and swept it over $\{2, 3, 4\}$. A gate of four tightened both counts rather than pinning the one it aims at, and only that arm ran to 998 of the trace's 999 steps, so this is a single comparison rather than a sweep. Both repairs fail: without a constraint on admission a paused session is readmitted on the next cycle, and with one the policy is a session cap again. The obstacle is granularity—`paused` takes effect at round boundaries, while pinning an instantaneous count needs control the permit mechanism does not have.

E. An independent adaptive controller

Every controller above is ours, which invites the objection that the negative result is a statement about one design rather than about adaptive admission. We therefore implemented a second, mechanistically different controller: an AIMD window over an agent-level admission set, following the control law of the congestion-based system we compare against in §VII [8], [22], acting on runtime gauges through the same named permits as Foyer. It does not beat the static front either, at either parameter setting (Table I): at the values reported in that paper's evaluation it finishes in 326.8 minutes at 79.4% SLO, 12.7 pp below the envelope, and with its retuned watermarks in 264.6 minutes at 52.8%, 34.3 pp below. This is our reimplementation of a published control law, not the authors' system, and a reimplementation can underperform its paper; we did not tune it beyond the two documented settings, and its first two attempts stalled because the launch path that ran it had no supervisor loop. Our own controller shows a smaller version of the same shape: one of the seventeen runs of ours in `stall.csv` has a window in which it admits nobody for at least 150 s while the engine has no running request and at least 85 sessions wait, with neither the SLO valve nor the hard stop holding admission—166.6 s, 0.93% of the span it

sampled; it cannot arise in our static caps, which have no admission decision to get stuck in.

VI. The Same Measurements on a Second Workload and a Second Accelerator

We repeated the same accounting, sweep and controller on a second replay—117 real Claude agent sessions, 390 rounds, same model, engine and 101,432-token pool—and the same sweep on an A100 (80 GB) replaying the main trace (nominal pool 983,728 tokens, 9.70× the 3090's). It ran entirely without a host tier—device-only points, envelope and controller—so it tests the negative result under a configuration that *cannot* separate the two.

a) The direction transfers; the level and the place do not.: Higher planned reuse (86.3% of prompt tokens against 75.5%), less of it realized (74.8% to 57.5% as the count rises, against 92.5% to 36.4%): the direction of §III survives, the sharpness does not, and we do not lean on slopes spanning ranges of different length and level. Nor does the *level* transfer: realization is *less* at 15.8% median device occupancy than the main trace's at a comparable count.

b) The tier mechanism moves with the tier.: The A100 criterion was fixed in advance on one decision axis: the $\overline{R}_{\text{busy}}$ at which realized reuse crosses 50%. On the 3090 that crossing is 1.7344; the A100 never crosses—at 3.4077, twice the 3090's, it is still at 92.5%. The pre-registered rule reads a crossing, not a slope, and returns *supported*. The same measurement in ϵ is flat, not steep—-0.74 on the first segment against the 3090's -0.21, then -0.27, $+0.03$, $+0.34$, $+0.72$, staying under one and reaching $+1.49$ on the last, a segment spanning 3.22 to 3.41 that carries the same small-denominator caveat we flag on the 3090: read it as the crossing, not as a rate. So added concurrency stops paying at roughly twice the running count on the larger-pool card—not the pool's factor of 9.7, and we do not claim it is; a supporting measurement, not a second criterion.

The best controller of §V there completes the replay—999 of 999 steps, no failures—in 88.3 minutes at a 100.0% deadline satisfaction rate. The sweep has since extended to caps eight and sixteen: cap eight finishes in 90.1 minutes, cap sixteen in 88.3—the same wall clock as the controller—with the front interpolated there 0.1 pp below the controller's SLO, inside the ± 1.0 pp band we judge in. The verdict is therefore a tie, obtained by *reaching* the controller's operating point rather than stopping short of it. *It is a weaker claim on this evidence*: *one* controller run, our pre-registered criterion for this tier has one axis and does not cover it, and the controller was rarely binding—its pool peaked at 52% utilisation and it admitted after a median wait of 1.6 s—so a tie is close to what "no admission decision at all" would give. What transfers is the mechanism; we do not read the tie as the negative result reproducing at full strength.

A. What this section does and does not buy

This buys a result that is not one trace family's: the same controller still does not beat the same kind of static cap on a second workload, and the tier mechanism behaves as the premise predicts on a second accelerator. It costs us the ability to *locate* the criterion's turn—the $\pm 10\%$ perturbation puts every segment's bounds across zero on both traces—and the controller sits below both turns; the second workload is closer to the regime where adaptive control *should* pay, and it still did not win there. Three things bound the hardware half: three of the seven A100 runs did not hold everything but the card fixed—cap one's pool 971,456 rather than 983,728, cap eight 943,541, cap sixteen 983,697—none pool-bound, so the shortfall does not reach the realization column; our no-control baseline is not clean—a foreign process in 82 of the run's 341 in-window samples, all of them in its last 23.8 minutes—so we report it as indicative, and co-tenancy inflates a baseline, so the direction is conservative; that run finished in 88.6 minutes at 99.7% SLO against the controller's 88.3; and the clean re-run we attempted afterwards did not produce one: pool 817,527 rather than 983,728, a median occupancy of 95.3%, the watcher records a foreign process in 566 of 566 in-window samples, 148.1 minutes at a 32.9% deadline satisfaction rate, and a per-request decode interval of 222.7 ms—a reading of a contended host, not a baseline, which we do not use for anything.

Machine	Pool	$\overline{R}_{\text{busy}}$	Occ.	Reuse	τ_2	SLO
3090-24G	101,432	0.9980	22.4%	92.5%	20.6	94.8
A100-80G	971,456	0.9997	2.1%	92.5%	14.3	99.9
3090-24G	101,432	1.6505	66.4%	60.7%	25.8	55.7
A100-80G	983,728	1.9702	6.5%	92.5%	14.5	100.0
3090-24G	101,432	1.8409	74.3%	36.4%	31.2	27.1
A100-80G	983,728	2.3356	8.0%	92.5%	15.5	100.0

TABLE II: The same static caps on two accelerators, rows paired at the same nominal cap. The A100's cap-one row came up with a pool of 971,456 rather than 983,728 (§VI-A), which does not affect its column. Occupancy is a share of the pool, τ_2 is in milliseconds, SLO is the share of first tokens within 10 s, and reuse is `reuse_tok` over the trace's own ceiling.

VII. Related Work

a) Related work.: Recent systems report large gains from treating an agent session rather than a request as the schedulable unit and preserving session state across tool calls [2], [3], [8], [9], [23]; one measures what losing that state costs—recomputation at 49.1% of end-to-end latency [8]—and we find the same pressure in our runs, whose mechanism our controller already has (§VIII). A second line improves the engine rather than the admission decision [5], [24]–[29]: our own measurements bound, without identifying, 3.3–8.8 ms of each 20.6–39.2 ms step—the overhead that line attacks; we expect it to move the frontier rather than decide who sits above it, but we did not run the interaction. A third keeps session state alive across the gaps we measure [20], [30]–[32] at finer control than our 2 s admission cycle, which bounds our negative result: we falsify one control granularity, not the fam-

ily. Serving an *application* also exposes dependencies between calls [33] and trace structure that request-level benchmarks miss [34], a structure our replay preserves but fixes in advance [35]—the limitation these works would press on.

b) Where the pressure comes from.: A parallel literature regulates the resource our mechanism turns on rather than the admissions that consume it: bandwidth reservation [36]–[38], its dynamic and per-bank variants [39]–[42]; on accelerators, memory extension across tiers [6], [7] and interference-aware scheduling [43]–[45]. None is a control we deploy.

Realized prefix reuse is a function of the admission policy and of the host tier, *planned* reuse a constant of the trace (80.9%); conflating the two got the direction wrong. The honest statement is not that admission cannot move this quantity—`cap4` alone moves it 25 points—but that the host tier keeps it from moving while the cap stays inside what the pool can hold: we varied admission policy, and eviction policy too—one `slru` run reads 80.5% against 80.6% for two runs of SGLang's [10] default and 80.87% planned, though it differs from both in card and launcher and from one in controller revision, so it cannot isolate the eviction policy's effect. A characterization of production KV-cache traffic finds eviction-policy design "highly workload-dependent" [46]—reason to expect one trace family to understate the space, though it is not a policy comparison and we do not present it as one.

VIII. Discussion

a) Why published gains do not reappear.: The systems cited above win by keeping session state alive across tool waits and pausing at agent granularity rather than request granularity; Foyer has both by construction (§II), so admission intelligence on top adds nothing we can resolve here; the arm above the front is a one-expression change to the *release* rule, above the line on two of its three runs (§V). The distinction we must not blur is between *choosing* to discard state and *having* it evicted: retention here is the engine's radix cache doing its ordinary job, not something the controller implements, and §III measures that the cache evicts parked prefixes under pressure and that they are recomputed. A system managing the host tier as an explicit policy—as [9], [12] and [30] do—holds a lever we do not, and our negative result does not cover it.

b) The threshold decides the verdict, and we report that.: Every SLO number here counts a request as good if its time-to-first-token is under 10 s—the whole vertical axis—so we re-derived the frontier at 5 s and 20 s from the same logs. The ordering of the static caps does not change, but where the controllers sit does: at the loose end six cross, `horizon=` 1 by $+1.5$, `horizon=` 2 by $+1.0$, the queue-scanning arm by $+1.4$, *its repeat by* $+2.4$ *and its third run by* $+1.2$, and `single-flight off`'s second run by $+2.0$. Our headline claim is conditional, and this is where the condition bites, in the direction that costs us most. At the strict threshold every other number here is judged at, the queue-scanning arm of §V is above the line on two of its three runs, by $+1.30$ and $+2.16$ pp, the third inside it at $+0.29$ pp, against the 0.5 to 4.6 pp that separates repeats of a single controller configuration; it is the only arm above the line at all three thresholds and the only one whose repeat confirms it—its third run does not. Three of the six that cross at the loose end are contradicted by their own repeats. What this supports is neither a verdict on admission control nor a single-run anomaly: a one-expression change to the *release* rule is above the static envelope on two of its three runs, while every arm that changes the *admission* decision is not.

c) The SLO metric excludes the controller's own cost, and we checked.: The runner measures time-to-first-token from the moment the request is *submitted to the engine*, so a permit wait is invisible—and that wait is precisely the cost an admission controller imposes. We recomputed all thirty-seven 200-session runs we measured on this trace on a ready-to-first-token basis, including the two anchor repeats that define the line. The correction is large—every run that gates admission loses 0.4–19.5 pp of SLO, the tightest caps queueing the hardest—but the direction of the verdict does *not* change: the seven that cross the ± 1.0 pp band all move toward the negative, and on this clock *no* run is above the line, the arm that clears it on the engine clock falling outside it on one of its three runs. We do not claim the verdict is unchanged under every SLO definition—only that moving the clock moves none from below the line to above it. We report the engine clock as primary because every other number in this paper uses it; the ready clock is the stricter of the two.

d) Limitations.: Our results come from one 7B model: we did test the workload axis once more (§VI) and the hardware axis on a second accelerator (§VI)—a static sweep plus one controller run, where the controller ties rather than crosses, and one row there we could not get clean—so the negative result *across the nine Foyer variants* is still measured on one card. Only nine configurations have been repeated, so the replicate spread in §V rests on one to three degrees of freedom per configuration, and a margin as small as ours would need several more repeats; we did not run them. The front every verdict is measured against is a piecewise interpolation between measured static points, not a measured policy (§II)—it is the stricter of the two comparisons, but *stricter* is not the same as *valid*: we have since run that policy once, landing 0.89 pp *below* the line at its own wall clock inside the tie band (§II, `slice23.csv`), and one point is not the whole segment, so we cannot exclude that it overstates what a static mix could deliver elsewhere on it; if it does, "no controller is above the line" would not by itself establish that there is no room. That is the single assumption this paper's verdict rests on, and it is an assumption. The pool holds about four median sessions, so concurrency is capacity-bound by construction, and on a device with a much larger pool the premise may not hold and the conclusion may invert. Both completed attempts at that baseline were taken under co-tenancy, so the one comparison we have there is indicative only, and the controller's edge over the no-control arm is the presence of *a* cap, not the quality of the choice. The load is stationary, single-tenant and single-SLO, so we cannot say what happens while they change, and the experiment that would settle that gives us no result—*the*

switcher never fired, so what the runs meant to test it measure is the run-to-run spread of one configuration and not evidence that adaptivity pays, and the baselines are unusable as controls. Multi-tenant and heterogeneous-SLO settings are where we would look next.

e) Reproducibility.: Front and signed distances are recomputed from the raw per-step logs over a full-precision run table; §IV from its own elasticity generator. Every number has a row in one of our data files, and five quantities this paper turns on are re-derived by five checks sharing no code with the generators, each reproducing the table it checks. The per-run readings, the full run table, the audit forensics and the controller revision history are collected in one supplementary document; *no verdict in this paper depends on it.*

Co-tenancy on the measurement host. An audit flags every run that overlapped three or more others for at least an hour: fourteen are, and the incident that created those windows killed four runs outright, none of which appears in our tables. The flag is a risk marker, not a measurement of damage: the engine-side counters are indistinguishable between affected and clean runs, which is why the audit runs on the client-side decode interval τ_2 [37]—where the damage is legible it is large, with τ_2 at 33.3 ms on the worst-affected arm when flagged against 24.8 ms on re-measurement, and 23.8–27.5 ms across seventeen clean runs of the same controller family. Four of the fourteen were damaged, have all been re-measured alone, and moved 4.5–11.2 pp without crossing the line they were measured against; their rows carry the clean numbers and no damaged row remains, and we keep the flag on the one undamaged row rather than quietly dropping it.

The controller artifact. The runs record four different hashes of our controller script, none of them the repository copy, so no single revision reproduces every number; *all four* runs recorded under one of those copies are flagged by the audit, so the revision and the co-tenancy cannot be separated by any comparison in our data. We state the confound rather than a resolution: the recorded parameters and build do match the repository, but the copy most of our arms ran under diverges in two places, so §V-B tests the fix plus a looser admission rate, not the fix as written, and the launcher hashes are ones we can neither diff nor certify.

Seven claims were retracted during this work, and we list them because the retractions were the most informative events: an audit whose valve predicate was inverted, so the "windows" we first reported were the valve doing its job; the denominator of the step cost, which divided by the whole-run mean running count while multiplying by busy time, making τ come out low by exactly the idle fraction (36% for `cap1`); a median-based reading of the tool wait that inverted a central assumption; an oracle-monotonicity claim that rested on an arm later shown invalid; a double-charging argument; a monotone idle-rate claim; and a refutation that was itself refuted. Each is reported with its evidence in `claims_audit.md`.

IX. Conclusion

We asked whether an adaptive admission controller can beat a tuned static concurrency cap on a memory-bound agentic LLM workload. On our system none of the nine variants we built beats it beyond the tie band, against the interpolated static front—the strictest of the two baselines we could have used (§II), since it sits above the discrete static points. One arm's readings are above the band on two of its three runs, and we say which: the same controller with one expression changed, by 1.3 and 2.2 pp—a change to the *release* rule rather than to the admission decision. We expected the reason to be a cancellation—admission policy sets how many requests are in flight, the per-step cost rises with that number, and the product is the wall clock—and our two-part decomposition makes it measurable: it holds, but on our configuration it is not the binding one, the elasticity staying below one across the operating range. They did not lose where concurrency was expensive, though that is not the same as showing they could have won elsewhere. What we can say is narrower than what we set out to say: on this hardware and workload the admission rules we could build and tune did not beat a static cap, the capacity estimate they act on is $1.79\times$ the occupancy the engine reports, and a structural idle component that no policy we built recovers at a profit remains.

The part we would carry elsewhere is not the negative result but the mechanism behind it: prefix reuse is bounded by the workload, and whether that bound survives added concurrency is set by the memory tier—an association across every configuration we ran rather than an intervention (§III). That claim is measured on one trace family, on a pair that ran on two cards a day apart; a same-card pairing returns the same verdict over a smaller gap, and the capacity control returns no realization figure at all. Two boundaries come with it: on a second workload both measurements survive in direction and neither in level, so neither an occupancy nor a concurrency threshold transfers; and the variable-load case, where a controller should have its best chance, is measured but not usable, its baselines having been taken under host co-tenancy—we report it as open rather than as a result.

References

[1] K. Zhu, M. Jacob, C. Ma, Y. Pan, S. Wang, A. Krishnamurthy, and B. Kasikci, "TraceLab: Characterizing coding agent workloads for LLM serving," 2026, preprint; this paper's trace source.

[2] H. Li, R. He, Q. Mang, Q. Zhang, H. Mao, X. Chen, H. Zhou, H. Zhang, A. Cheung, J. Gonzalez, and I. Stoica, "Continuum: Efficient and robust multi-turn LLM agent scheduling with KV cache time-to-live," 2025, preprint, no venue.

[3] U. Meng, U. Zhao, D. Li, G. Gao, and F. Du, "Talaria: Session-aware serverless serving of hundred-billion-parameter LLMs," 2026, preprint; author names as listed on arXiv.

[4] B. Zheng, A. H. Yang, J. Ruan, J. He, X. Li, Y.-H. Chen, Z. Wang, and X. Ma, "Real-time language model jamming: A case study for live music accompaniment generation," in *2026 IEEE 32nd Real-Time and Embedded Technology and Applications Symposium (RTAS)*, 2026, pp. 261–274.

[5] W. Kwon, Z. Li, S. Zhuang, Y. Sheng, L. Zheng, C. H. Yu, J. E. Gonzalez, H. Zhang, and I. Stoica, "Efficient memory management for large language model serving with PagedAttention," in *Proceedings of the 29th ACM Symposium on Operating Systems Principles (SOSP '23)*, 2023, pp. 611–626.

[6] W. Kang, F. Muzzini, G. Brilli, J.-C. Kim, J. Lee, and H. S. Chwa, "ZeroSwap: Minimizing swap overhead for real-time multi-DNN inference via SSD-based GPU memory extension," in *2026 IEEE 32nd Real-Time and Embedded Technology and Applications Symposium (RTAS)*, 2026, pp. 234–246.

[7] W. Kang, J. Lee, Y. Lee, S. Oh, K. Lee, and H. S. Chwa, "RT-Swap: Addressing GPU memory bottlenecks for real-time multi-DNN inference," in *2024 IEEE 30th Real-Time and Embedded Technology and Applications Symposium (RTAS)*, 2024, pp. 373–385.

[8] Q. Chen, Z. Ye, T. Tang, P. Sun, B. Tian, G. Wang, S. Li, Y. Wen, Z. Han, and T. Zhang, "CONCUR: High-throughput agentic batch inference of LLM via congestion-based concurrency control," in *International Conference on Machine Learning (ICML 2026)*, 2026, poster, Seoul, July 2026; OpenReview id U7c6glPdDW; arXiv:2601.22705.

[9] T. Xia, H. Li, Z. Li, X. Chen, H. Kang, Y. Qiao, Y. Xu, and I. Stoica, "Idleness is relative: Exploiting tool-call idle windows for offloading in agentic systems with MORI," 2026, preprint, no venue.

[10] L. Zheng, L. Yin, Z. Xie, C. Sun, J. Huang, C. H. Yu, S. Cao, C. Kozyrakis, I. Stoica, J. E. Gonzalez, C. Barrett, and Y. Sheng, "SGLang: Efficient execution of structured language model programs," in *Advances in Neural Information Processing Systems 37 (NeurIPS 2024)*, 2024, pp. 62 557–62 583, arXiv:2312.07104; v1 title was "Efficiently Programming Large Language Models using SGLang".

[11] B. Hui, J. Yang, Z. Cui, J. Yang, D. Liu, L. Zhang, T. Liu, J. Zhang, B. Yu, K. Lu, K. Dang, Yang Fan, Yichang Zhang, A. Yang, R. Men, F. Huang, Bo Zheng, Yibo Miao, S. Quan, Yunlong Feng, X. Ren, X. Ren, J. Zhou, and J. Lin, "Qwen2.5-coder technical report," 2024.

[12] Y. Sheng, L. Zheng, B. Yuan, Z. Li, M. Ryabinin, B. Chen, P. Liang, C. Ré, I. Stoica, and C. Zhang, "FlexGen: High-throughput generative inference of large language models with a single GPU," in *Proceedings of the 40th International Conference on Machine Learning (ICML 2023)*, ser. Proceedings of Machine Learning Research, vol. 202. PMLR, 2023, pp. 31 094–31 116.

[13] M. Lumme, M. Roitzsch, and A. Lackorzyński, "Scheduling constraints: A universal OS mechanism for managing shared resources," in *2026 IEEE 32nd Real-Time and Embedded Technology and Applications Symposium (RTAS)*, 2026, pp. 400–413.

[14] C. L. Liu and J. W. Layland, "Scheduling algorithms for multiprogramming in a hard-real-time environment," *Journal of the ACM*, vol. 20, no. 1, pp. 46–61, 1973.

[15] R. Wilhelm, J. Engblom, A. Ermedahl, N. Holsti, S. Thesing, D. Whalley, G. Bernat, C. Ferdinand, R. Heckmann, T. Mitra, F. Mueller, I. Puaut, P. Puschner, J. Staschulat, and P. Stenström, "The worst-case execution-time problem—overview of methods and survey of tools," *ACM Transactions on Embedded Computing Systems*, vol. 7, no. 3, pp. 1–53, 2008.

[16] J. D. C. Little, "A proof for the queuing formula: $l = \lambda w$," *Operations Research*, vol. 9, no. 3, pp. 383–387, 1961.

[17] B. Ye, F. Marković, and B. B. Brandenburg, "Framework-agnostic model inference for intra-thread real-time tasks," in *2026 IEEE 32nd Real-Time and Embedded Technology and Applications Symposium (RTAS)*, 2026, pp. 68–81.

[18] Z. Zhang, Y. Sheng, T. Zhou, T. Chen, L. Zheng, R. Cai, Z. Song, Y. Tian, C. Ré, C. Barrett, Z. Wang, and B. Chen, "H2O: Heavy-hitter oracle for efficient generative inference of large language models," in *Advances in Neural Information Processing Systems 36 (NeurIPS 2023)*, 2023, pp. 34 661–34 710.

[19] Z. Liu, A. Desai, F. Liao, W. Wang, V. Xie, Z. Xu, A. Kyrillidis, and A. Shrivastava, "Scissorhands: Exploiting the persistence of importance hypothesis for LLM KV cache compression at test time," in *Advances in Neural Information Processing Systems 36 (NeurIPS 2023)*, 2023, pp. 52 342–52 364.

[20] B. Gao, Z. He, P. Sharma, Q. Kang, D. Jevdjic, J. Deng, X. Yang, Z. Yu, and P. Zuo, "Cost-efficient large language model serving for multi-turn conversations with CachedAttention," in *2024 USENIX Annual Technical Conference (USENIX ATC '24)*. USENIX Association, 2024, pp. 111–126.

[21] P. J. Denning and J. P. Buzen, "The operational analysis of queueing network models," *ACM Computing Surveys*, vol. 10, no. 3, pp. 225–261, 1978.

[22] V. Jacobson, "Congestion avoidance and control," in *Symposium Proceedings on Communications Architectures and Protocols (SIGCOMM '88)*, 1988, pp. 314–329.

[23] H. Kang, Z. Li, W. Xu, X. Yang, Y. Chen, J. Wang, B. Chen, T. Krishna, C. Xu, and S. Arora, "ThunderAgent: A simple, fast and program-aware agentic inference system," 2026, preprint, no venue.

[24] G.-I. Yu, J. S. Jeong, G.-W. Kim, S. Kim, and B.-G. Chun, "Orca: A distributed serving system for transformer-based generative models," in *16th USENIX Symposium on Operating Systems Design and Implementation (OSDI '22)*. USENIX Association, 2022, pp. 521–538.

[25] A. Agrawal, N. Kedia, A. Panwar, J. Mohan, N. Kwatra, B. Gulavani, A. Tumanov, and R. Ramjee, "Taming Throughput-Latency tradeoff in LLM inference with Sarathi-Serve," in *18th USENIX Symposium on Operating Systems Design and Implementation (OSDI '24)*. USENIX Association, 2024, pp. 117–134.

[26] Y. Zhong, S. Liu, J. Chen, J. Hu, Y. Zhu, X. Liu, X. Jin, and H. Zhang, "DistServe: Disaggregating prefill and decoding for goodput-optimized large language model serving," in *18th USENIX Symposium on Operating Systems Design and Implementation (OSDI '24)*. USENIX Association, 2024, pp. 193–210.

[27] R. Qin, Z. Li, W. He, J. Cui, H. Tang, F. Ren, T. Ma, S. Cai, Y. Zhang, M. Zhang, Y. Wu, W. Zheng, and X. Xu, "Mooncake: A KVCache-centric disaggregated architecture for LLM serving," *ACM Transactions on Storage*, vol. 22, no. 4, pp. 1–38, 2026.

[28] T. Dao, D. Fu, S. Ermon, A. Rudra, and C. Ré, "FlashAttention: Fast and memory-efficient exact attention with IO-awareness," in *Advances in Neural Information Processing Systems 35 (NeurIPS 2022)*, 2022, pp. 16 344–16 359.

[29] P. Patel, E. Choukse, C. Zhang, A. Shah, Í. Goiri, S. Maleki, and R. Bianchini, "Splitwise: Efficient generative LLM inference using phase splitting," in *2024 ACM/IEEE 51st Annual International Symposium on Computer Architecture (ISCA)*, 2024, pp. 118–132.

[30] L. Yu, J. Lin, and J. Li, "Stateful large language model serving with Pensieve," in *Proceedings of the Twentieth European Conference on Computer Systems (EuroSys '25)*, 2025, pp. 144–158.

[31] B. Sun, Z. Huang, H. Zhao, W. Xiao, X. Zhang, Y. Li, and W. Lin, "Llumnix: Dynamic scheduling for large language model serving," in *18th USENIX Symposium on Operating Systems Design and Implementation (OSDI '24)*. USENIX Association, 2024, pp. 173–191.

[32] B. Wu, Y. Zhong, Z. Zhang, S. Liu, F. Liu, Y. Sun, G. Huang, X. Liu, and X. Jin, "FastServe: Iteration-level preemptive scheduling for large language model inference," in *23rd USENIX Symposium on Networked Systems Design and Implementation (NSDI '26)*. USENIX Association, 2026, pp. 57–74.

[33] C. Lin, Z. Han, C. Zhang, Y. Yang, F. Yang, C. Chen, and L. Qiu, "Parrot: Efficient serving of LLM-based applications with semantic variable," in *18th USENIX Symposium on Operating Systems Design and Implementation (OSDI '24)*. USENIX Association, 2024, pp. 929–945.

[34] M. Wang, Y. Yue, S. Li, Y. E. Zhou, C. Wang, and J. Huang, "Benchmarking LLM serving systems for agentic AI workloads with XPerf," 2026, preprint.

[35] G. A. Bondar, A. Eisenklam, Y. Cai, R. Gifford, T. Sial, L. T. X. Phan, and A. Halder, "Generative profiling for soft real-time systems and its applications to resource allocation," in *2026 IEEE 32nd Real-Time and Embedded Technology and Applications Symposium (RTAS)*, 2026, pp. 289–302.

[36] H. Yun, G. Yao, R. Pellizzoni, M. Caccamo, and L. Sha, "MemGuard: Memory bandwidth reservation system for efficient performance isolation in multi-core platforms," in *2013 IEEE 19th Real-Time and Embedded Technology and Applications Symposium (RTAS)*, 2013, pp. 55–64.

[37] ——, "Memory bandwidth management for efficient performance isolation in multi-core platforms," *IEEE Transactions on Computers*, vol. 65, no. 2, pp. 562–576, 2016.

[38] A. Zuepke, A. Bastoni, W. Chen, M. Caccamo, and R. Mancuso, "MemPol: Policing core memory bandwidth from outside of the cores," in *2023 IEEE 29th Real-Time and Embedded Technology and Applications Symposium (RTAS)*, 2023, pp. 235–248.

[39] A. Agrawal, R. Mancuso, R. Pellizzoni, and G. Fohler, "Analysis of dynamic memory bandwidth regulation in multi-core real-time systems," in *2018 IEEE Real-Time Systems Symposium (RTSS)*, 2018, pp. 230–241.

[40] C. Sullivan, A. Manley, M. Alian, and H. Yun, "Per-bank bandwidth regulation of shared last-level cache for real-time systems," in *2024 IEEE Real-Time Systems Symposium (RTSS)*, 2024, pp. 336–348.

[41] A. Zuepke, A. Pradhan, D. Ottaviano, A. Bastoni, and M. Caccamo, "ETM^2: Empowering traditional memory bandwidth regulation using

ETM," in *2026 IEEE 32nd Real-Time and Embedded Technology and Applications Symposium (RTAS)*, 2026, pp. 180–193.

[42] C. R. Sullivan, A. Mamandipoor, C. R. Strickler, and H. Yun, "Per-bank memory bandwidth regulation for predictable and performant real-time systems," in *2026 IEEE 32nd Real-Time and Embedded Technology and Applications Symposium (RTAS)*, 2026, pp. 167–179.

[43] Y. Xiang and H. Kim, "Pipelined data-parallel CPU/GPU scheduling for multi-DNN real-time inference," in *2019 IEEE Real-Time Systems Symposium (RTSS)*, 2019, pp. 392–405.

[44] F. Aromolo, N. Salamini, J. Del Granchio, A. Biondi, M. Marinoni, and G. Buttazzo, "Time-predictable acceleration of deep neural networks on FPGA SoCs with multi-core DPUs," in *2026 IEEE 32nd Real-Time and Embedded Technology and Applications Symposium (RTAS)*, 2026, pp. 275–288.

[45] B. Sun, J. Li, T. Kloda, T. Abdelzaher, and M. Caccamo, "AI inference in the heat: Thermal-aware strict partitioning for configurable real-time gang tasks," in *2026 IEEE 32nd Real-Time and Embedded Technology and Applications Symposium (RTAS)*, 2026, pp. 247–260.

[46] J. Wang, J. Han, X. Wei, S. Shen, D. Zhang, C. Fang, R. Chen, W. Yu, and H. Chen, "KVCache cache in the wild: Characterizing and optimizing KVCache cache at a large cloud provider," in *2025 USENIX Annual Technical Conference (USENIX ATC '25)*. USENIX Association, 2025, pp. 465–482.